

Tower Defense **Estocástico**

Guion por integrante

Para cada slide: su **referente/lead**, los **puntos a mencionar**, un **apoyo teórico/práctico** sobre los conceptos usados en esa diapositiva, y un **“Apoyo material estudio”** con los conceptos clave de la materia (Unidad I — simulación y modelos · Unidad II — números pseudoaleatorios · Unidad III — teoría de colas).

● **Luciano Esteban** 01·02·03·04·05·22

● **Thaiel Anaya** 06·07·08·09·10

● **Martín Blasson** 11·12·13·21

● **Luca Sucri** 14·15·16·17·18·19·20

Presentación en equipo: la charla es colaborativa — todos intervenimos cuando haga falta. El **referente/lead** de cada slide (marcado abajo) es quien la conduce, no el único que habla. El **cierre (slide 22) lo hace Luciano**.

Ing. en Informática · UCEMA · Prof. Sergio Sirotinsky | Simulación de Sistemas | Modelo (M/M/c)
(FIFO/K/∞) · $\lambda=0.40$ · $\mu=0.25$ · $c=3$ · $K=10$

Luciano Esteban

Abre y cierra la charla. Slides **01 Portada** · **02 Recorrido** · **03 Panorama** · **04 Problema** · **05 Modelo de colas** · **22 Cierre**. Marca el encuadre (qué se entrega y por qué), da el panorama de la máquina de la demo y la traducción del juego a teoría de colas.

SLIDE 01 · PORTADA Tower Defense Estocástico

Referente: Luciano

► PUNTOS A PRESENTAR

- Presentar al equipo y el título. Frase ancla: **“el entregable no es un videojuego, es la validación de un modelo de colas”**.
- Anticipar que hay **demo en vivo** al final: un Tower Defense real que reproduce la simulación.
- Marca de la casa: tono descontracturado, intervenimos cuando haga falta.

▼ Apoyo teórico / práctico

Modelo **UNIDAD I** — una representación simplificada de un sistema real que conserva las relaciones relevantes para responder una pregunta. Acá el sistema real es “defender una base de oleadas”; el modelo es una red de líneas de espera.

Simulación **UNIDAD I** — experimentar sobre el modelo (no sobre el sistema real) para estimar su comportamiento. Sirve cuando el sistema es estocástico y no hay fórmula cerrada exacta.

Estocástico — gobernado por el azar: las llegadas de enemigos y los tiempos de ataque son variables aleatorias, no valores fijos. De ahí “Tower Defense **Estocástico**”.

★ Apoyo material estudio

- **Vocabulario base** **UNIDAD I** — **sistema** (entidades que interactúan), **entidad** (enemigo, torre), **atributo**, **estado** (lo que describe al sistema en un instante) y **evento** (cambio de estado).
- **Clasificación de modelos** **UNIDAD I** — determinístico vs. **estocástico**, estático vs. **dinámico**, continuo vs. **discreto**. Este TP es matemático, dinámico, estocástico y de eventos discretos.
- **¿Cuándo simular?** — cuando el sistema es complejo o estocástico, no hay fórmula cerrada exacta, o experimentar en la realidad es caro/riesgoso.

► PUNTOS A PRESENTAR

- Dar el índice como "oleadas": **W1** problema y modelo · **W2** aleatoriedad y motor · **W3** validación y resultados · **W4** experimentos, arquitectura y demo.
- Aclarar que cada Wave la lleva principalmente un integrante, pero el tono es de equipo.
- Recordar el cierre con demo en vivo.

▼ Apoyo teórico / práctico

Por qué este orden — sigue el método de un estudio de simulación: (1) formular el problema, (2) construir el modelo y sus entradas aleatorias, (3) **verificar y validar**, (4) experimentar y concluir. Es el esqueleto clásico de la materia llevado a un caso lúdico.

Validación vs. verificación — **verificar** es "¿programé bien el modelo?"; **validar** es "¿el modelo representa la realidad?". Las Waves 3 y 4 son justamente eso.

★ Apoyo material estudio

- **Etapas de un estudio de simulación** **UNIDAD I** — formular el problema → datos → modelo conceptual → programa → **verificación** → **validación** → diseño de experimentos → corridas → análisis → documentar.
- **Verificación vs. validación** — verificar = el programa hace lo que el modelo dice; validar = el modelo representa la realidad.
- **Variable de estado** — describe el sistema en cada instante (longitud de cola, temperatura de cada torre).

► PUNTOS A PRESENTAR

- Dar el **panorama** antes del detalle: lo que se verá en vivo se arma en **tres pasos**, y uno solo ocurre en pantalla.
- **Paso 1 · offline:** Python + SimPy simula la batalla por **eventos discretos** y arma una línea de tiempo completa, **una sola vez**.
- **Paso 2 · contrato:** exporta `output.json` — una **foto del sistema por instante** (cola, fugas, estado de cada torre); con **semilla fija** es reproducible.
- **Paso 3 · en vivo:** Godot **lee e interpola**; el HUD dibuja la teoría encima. La pantalla **no calcula nada**.
- Anticipar el **selector**: en la demo cambiamos de escenario (λ , c, FIFO/Prioridad) en caliente y el juego recarga el JSON al instante.

▼ Apoyo teórico / práctico

Separación cálculo / visualización — el motor que resuelve la dinámica estocástica (pesado) se separa del visor que sólo reproduce (liviano). Es la misma idea que retoma Luca en Arquitectura, vista acá **de un vistazo**.

Simulación por eventos discretos **UNIDAD III** — el reloj salta de evento a evento (arribo, fin de servicio, sobrecalentamiento); de ahí sale la línea de tiempo que se guarda y luego se reproduce.

Reproducibilidad por la semilla **UNIDAD II** — al ser pseudoaleatorio, misma **semilla** $\Rightarrow$ **misma corrida** $\Rightarrow$ **mismo** `output.json` : lo que mostramos es auditable y se puede repetir igual.

Por qué importa el encuadre: deja claro desde el arranque que el entregable es la **simulación validada**; el juego es la forma de verla.

★ Apoyo material estudio

- **Modelo $\rightarrow$ programa $\rightarrow$ animación** **UNIDAD I** — etapas de un estudio de simulación: se construye el modelo, se lo programa (verificación) y la **animación** ayuda a comunicarlo y a detectar errores.
- **Estado del sistema** **UNIDAD I** — lo que describe al sistema en un instante (cola, temperatura de cada torre); el `output.json` guarda justamente esos estados muestreados.
- **Eventos discretos + variable continua** **UNIDAD III** — llegadas/servicios son eventos; la temperatura es continua. La demo reproduce ambos sin recalcular.

► PUNTOS A PRESENTAR

- Describir el sistema: enemigos avanzan → zona con **c torres** los ataca → si están ocupadas, **esperan en cola**.
- Punto clave: si la zona está **llena**, el enemigo **se fuga** y daña la base (esto es lo que después llamamos "bloqueo").
- Repetir el encuadre: el entregable es la **validación del modelo**, el juego es el vehículo.
- Apoyarse en el diagrama spawn → cola(FIFO) → c torres → base.

▼ Apoyo teórico / práctico

Sistema de líneas de espera (colas) **UNIDAD III** — clientes que llegan, esperan si el servicio está ocupado, son atendidos y se van. Sus tres piezas: **proceso de llegadas**, **mecanismo de servicio** y **disciplina de la cola**.

Mapeo del juego — enemigo = **cliente**; torre = **servidor**; "esperar a que se libere una torre" = **cola**; "fugarse porque está lleno" = **pérdida/bloqueo** por capacidad finita.

Capacidad finita K — el sistema admite a lo sumo K enemigos entre cola y servicio; el K-ésimo+1 se pierde. Es lo que hace que existan fugas y que el modelo sea "con pérdida".

★ Apoyo material estudio

- **Estructura de un sistema de colas** **UNIDAD III** — **fuente/población**, **proceso de llegadas**, **cola** (capacidad K + disciplina), **servidores** (c) y **salida**.
- **Población infinita vs. finita** — acá se asume población infinita (∞): la tasa de llegada no depende de cuántos ya estén dentro.
- **Sistema con pérdida (bloqueo)** — si la capacidad es finita, el cliente que llega lleno se pierde; es la **fuga** del juego.

► PUNTOS A PRESENTAR

- Mostrar la tabla de equivalencias (enemigo↔cliente, torre↔servidor, fuga↔bloqueo).
- Leer la notación **(M/M/c)(FIFO/K/∞)** en voz alta y desglosarla.
- Introducir $\rho = \lambda / (c \cdot \mu)$ y la condición de estabilidad $\rho < 1$ — es el concepto que sostiene toda la Wave 3.

▼ Apoyo teórico / práctico

Notación de Kendall-Lee UNIDAD III — formato **(A/B/c)(disciplina/K/N)** : **A** distribución de llegadas, **B** de servicio, **c** n° de servidores, **disciplina** de atención, **K** capacidad del sistema, **N** tamaño de la población.

M = markoviano — llegadas Poisson y servicios exponenciales (sin memoria). **c** torres, disciplina **FIFO** (primero en llegar, primero en ser atendido), **K** finita, población ∞ .

$$\rho = \lambda / (c \cdot \mu) \rightarrow \text{estable si } \rho < 1$$

Factor de utilización ρ — fracción del tiempo que, en promedio, está ocupado cada servidor. Si $\rho \geq 1$ la cola crece sin límite (en capacidad infinita): el sistema “no da abasto”.

$$\lambda = 0.40 \text{ arribos/s}$$

$$\mu = 0.25 \text{ serv/s}$$

$$c = 3$$

$$\rho = 0.533$$

★ Apoyo material estudio

- **Notación de Kendall** UNIDAD III — **A/B/c/K/N/disciplina** : distribución de llegadas / de servicio / n° de servidores / capacidad / población / orden de atención.
- **Proceso de Poisson ↔ Exponencial** — si las llegadas son Poisson de tasa λ , los **tiempos entre llegadas** son $\text{Exp}(\lambda)$. Igual para el servicio con μ .
- **Tasas e intensidades** — λ (arribos/u.t.), μ (servicio/u.t.), intensidad ofrecida $a = \lambda / \mu$ (Erlangs), utilización $\rho = \lambda / (c \cdot \mu)$.
- **Estabilidad** — con cola infinita se requiere $\rho < 1$; con K finita el sistema nunca “explota”, pero aparece bloqueo.

► PUNTOS A PRESENTAR

- Cerrar con la frase: “**defendemos el modelo, no el juego — pero el juego lo hace divertido**”.
- Mostrar los 4 repos (RAG/doc, backend, frontend, presentación) e invitar a preguntas.
- Quedar para las **preguntas** del profesor — la demo ya ocurrió en la slide 20, el cierre es sólo el final.

▼ Apoyo teórico / práctico

Mensaje a dejar grabado — el TP muestra el ciclo completo: modelar un sistema estocástico, generarlo con números pseudoaleatorios, validarlo contra la teoría de colas y usar la simulación para responder preguntas de diseño (¿cuántas torres?).

Posibles preguntas y cómo pararse — “¿por qué simular y no usar la fórmula?” → porque la **temperatura** rompe el supuesto markoviano (ver W2). “¿cómo sé que está bien?” → por la **Ley de Little** y el acuerdo con M/M/c sin temperatura (ver W3).

★ Apoyo material estudio

- **Mapa de la materia cubierto** — **Unidad I** (sistema/modelo/simulación, V&V) · **Unidad II** (PRNG, transformada inversa, v.a.) · **Unidad III** (colas Kendall, Little, M/M/1, M/M/c, M/M/c/K, eventos discretos).
- **Glosario exprés para preguntas** — **p**: utilización (fracción de tiempo ocupado); **Little** $L=\lambda W$: cantidad media = tasa × tiempo medio; **Erlang B**: prob. de bloqueo / **Erlang C**: prob. de esperar; **FIFO**: se atiende por orden de llegada; **bloqueo/pérdida**: el que llega con el sistema lleno se va; **estacionario**: el sistema ya “se asentó”; **transformada inversa**: generar una v.a. a partir de un $R \sim U(0,1)$.
- **Toma de decisiones con simulación** **UNIDAD I** — el fin no es el número sino elegir bien (cuántas torres, cómo priorizar) bajo incertidumbre.

Thaiel Anaya

Wave 02 — aleatoriedad y motor. Slides **06 Variables · 07 Aleatoriedad · 08 Motor SimPy · 09 Código SimPy I · 10 Código SimPy II.** Explica de qué está hecho el modelo, cómo se generan los números aleatorios, por qué hace falta simular y cómo se ve eso en el código.

► PUNTOS A PRESENTAR

- Clasificar las variables: **aleatorias** (arribos $\text{Exp}(\lambda)$, servicio $\text{Exp}(\mu)$, tipo de enemigo), **continua** (temperatura) y de **control/salida** (λ, μ, c, K , semilla $\rightarrow Lq, Wq, \rho$, fuga).
- Destacar la **temperatura** como “la variable continua del TP” y el elemento de juego.
- Cerrar con la **Ley de Little** como chequeo de consistencia.

▼ Apoyo teórico / práctico

Variable aleatoria discreta vs. continua **UNIDAD II** — discreta toma valores contables (tipo de enemigo); continua toma cualquier valor en un intervalo (temperatura, tiempos). Cada una se genera distinto.

Distribución exponencial **UNIDAD II/III** — modela el **tiempo entre eventos** de un proceso sin memoria; media $1/\lambda$ (arribos) y $1/\mu$ (servicio). Su “falta de memoria” es lo que hace markoviano al sistema.

Variable continua del TP — temperatura — evoluciona por una ecuación diferencial por tramos: rampa al disparar y **decaimiento exponencial hacia la temperatura ambiente** al descansar (esto último es la ley de enfriamiento de Newton). No es aleatoria: es **determinística dado el historial** de disparos.

Ley de Little: $L = \lambda \cdot W$ $Lq = \lambda \cdot Wq$

Ley de Little **UNIDAD III** — relación universal: el nº promedio de clientes en el sistema (L) es la tasa de llegadas efectiva por el tiempo promedio en el sistema (W). Se usa como control: si los números simulados la cumplen, hay consistencia interna.

★ Apoyo material estudio

- **V.A. discreta vs. continua** **UNIDAD II** — discreta: valores contables (tipo de enemigo, función de probabilidad); continua: intervalo real (tiempos, temperatura, función de densidad $f(x)$).
- **Distribución exponencial** **UNIDAD II** — $f(x) = \lambda e^{-\lambda x}$, media $1/\lambda$, varianza $1/\lambda^2$; **sin memoria** (lo pasado no afecta lo que falta).
- **Ley de Little** **UNIDAD III** — $L = \lambda W$ vale para cualquier sistema estable; relaciona cantidad media y tiempo medio.
- **Poisson (discreta)** — cuenta de eventos en un intervalo; es la cara discreta del proceso cuyos tiempos son exponenciales.

► PUNTOS A PRESENTAR

- Explicar el generador **congruencial lineal** y por qué “pseudo”: es determinístico pero parece aleatorio.
- Resaltar las propiedades buscadas: **uniforme, independiente, reproducible (semilla), período amplio**.
- Mostrar la **transformada inversa** de la exponencial: cómo se pasa de un $R \sim U(0,1)$ a un tiempo.
- Aclarar la doble capa: el apunte (U2) es el **congruencial** —implementado en `prng.py`—; la corrida usa el generador por defecto del lenguaje + **streams independientes** (arribos/servicio/tipo).

▼ Apoyo teórico / práctico

Generador congruencial lineal (GCL) **UNIDAD II** — produce una sucesión de enteros con una recurrencia; dividiendo por m se obtienen uniformes en $[0,1)$.

$$X_{(i+1)} = (a \cdot X_i + C) \bmod m \quad R_i = X_i / m$$

Parámetros — **a** multiplicador, **C** incremento, **m** módulo, **X₀** semilla. La elección de a , C , m define el período (cuántos valores antes de repetir) y la calidad estadística.

Pseudoaleatorio — la sucesión es 100% determinística: misma **semilla** $\Rightarrow$ **misma corrida**. Esto es una ventaja: hace la simulación **reproducible y auditable** (mismo `output.json`).

Método de la transformada inversa **UNIDAD II** — para generar una v.a. con función de distribución F : se sortea $R \sim U(0,1)$ y se despeja $x = F^{-1}(R)$.

$$F(x) = 1 - e^{(-\lambda x)} = R \Rightarrow x = -(1/\lambda) \cdot \ln(R)$$

Generador del lenguaje (aclaración) — la corrida usa el PRNG por defecto de Python/NumPy, de período muy largo (se llama **Mersenne Twister**). **No es del apunte**: lo que se ve en U2 es el **congruencial**, implementado en `prng.py`; este es solo el generador que trae el lenguaje. Usar **streams independientes** (uno por fuente de azar) evita correlaciones.

★ Apoyo material estudio

- **Propiedades de buenos números aleatorios** **UNIDAD II** — **uniformidad** e **independencia** (y, en la práctica, período largo y reproducibilidad).
- **Generador congruencial lineal** **UNIDAD II** — $X_{(i+1)} = (aX_i + C) \bmod m$; mixto ($C \neq 0$) vs. multiplicativo ($C = 0$). El período máximo depende de a , C , m .
- **Pruebas estadísticas** **UNIDAD II** — de **uniformidad** (chi-cuadrado, Kolmogórov-Smirnov) y de **independencia** (prueba de corridas).
- **Métodos de generación de v.a.** **UNIDAD II** — **transformada inversa** (la del TP), aceptación-rechazo, convolución. Exponencial: $x = -(1/\lambda) \cdot \ln(1-R)$.

► PUNTOS A PRESENTAR

- Explicar el motor: cola FIFO con `simpy.Store`, cada torre es un proceso que la consume.
- **K finita** → el que llega con el sistema lleno se fuga (bloqueo).
- Idea central de la Wave: la **temperatura rompe el supuesto markoviano** (las c torres ya no están siempre disponibles) → la fórmula cerrada queda como **cota** y por eso hay que **simular**.
- **Cue de demo:** mirar las torres pasar de azul a rojo y apagarse en cooldown.

▼ Apoyo teórico / práctico

Simulación por eventos discretos (DES) **UNIDAD III** — el reloj salta de evento a evento (arribo, fin de servicio, sobrecalentamiento...), no avanza en pasos fijos. SimPy es una librería de DES basada en **procesos** y **recursos**.

Recurso / servidor — cada torre modela un servidor que toma un enemigo de la `Store` (cola), lo “atiende” un tiempo $\text{Exp}(\mu)$ y lo libera.

Supuesto markoviano **UNIDAD III** — las fórmulas M/M/c suponen **c servidores siempre disponibles** y tasas constantes. Si una torre entra en cooldown, la capacidad efectiva baja y deja de cumplirse: el sistema ya no es M/M/c puro.

Por eso simular > fórmula: la expresión analítica da una cota optimista; sólo la simulación captura el efecto de la indisponibilidad térmica sobre la fuga real.

★ Apoyo material estudio

- **Mecánica de eventos discretos** **UNIDAD III** — **reloj de simulación, lista de eventos futuros (FEL)** y avance al próximo evento (next-event time advance).
- **Enfoques de modelado** — por **eventos**, por **procesos** (el de SimPy) o por **actividades**.
- **Estadísticas** — contadores **de tiempo** (área bajo $L(t)$) y **de conteo** (n° de fugas), de donde salen L , L_q , W_q .
- **Proceso de nacimiento-muerte** — base markoviana de M/M/c: el estado es el n° de clientes; nacer = llegar, morir = terminar servicio.

► PUNTOS A PRESENTAR

- Mostrar que **cada línea es un concepto** de la materia, no “código suelto”.
- `arrivals() : env.timeout(Exp( $\lambda$ ))` = cuándo llega el próximo (**Poisson**); si el sistema está lleno (capacidad **K**), se cuenta como **fuga/bloqueo**; si hay lugar, entra a la cola **FIFO**.
- `tower_proc()` : cada torre **toma** el siguiente de la cola (espera si está vacía) y lo atiende un tiempo **Exp(μ)**.
- Mensaje: el programa **implementa** exactamente (M/M/c)(FIFO/K/ ∞).

▼ Apoyo teórico / práctico

Procesos y recursos en SimPy UNIDAD III — el modelo se arma como **procesos** (las llegadas, cada torre) que comparten una **cola** (Store FIFO). `yield env.timeout(t)` hace avanzar el reloj hasta el próximo evento.

Llegadas Poisson $\leftrightarrow$ tiempos Exp(λ) UNIDAD II/III — sortear el tiempo hasta el próximo arribo con `Exp( $\lambda$ )` es generar un proceso de llegadas Poisson de tasa λ .

Bloqueo por capacidad K UNIDAD III — el `if in_system() >= K` es la traducción literal de “sistema con pérdida”: el cliente que llega lleno se va = la **fuga**.

★ Apoyo material estudio

- **Enfoque por procesos** UNIDAD III — describir el sistema como entidades que “viven” un ciclo (llegar $\rightarrow$ encolar $\rightarrow$ ser atendido $\rightarrow$ salir); SimPy lo resuelve por eventos discretos por debajo.
- **Disciplina FIFO** UNIDAD III — la Store entrega en orden de llegada: primero en entrar, primero en ser atendido.
- **Servicio Exp(μ)** UNIDAD II — el tiempo de “disparo” se sortea por **transformada inversa** de la exponencial a partir de un $R \sim U(0,1)$.

► PUNTOS A PRESENTAR

- `temp_at()` es la **variable continua** de estado: **rampa** al disparar y **decaimiento exponencial** hacia la temperatura ambiente al descansar (ley de enfriamiento de Newton).
- Al superar `T_max` la torre entra en **cooldown** y deja de atender → **baja la capacidad efectiva** y se rompe el supuesto markoviano.
- `run()` arma el motor: lanza el proceso de **llegadas** y un proceso por **torre**; ese es el "esqueleto" de la simulación.
- Enlazar con la idea de la Wave: **por esto hay que simular**, la fórmula sola no ve la temperatura.

▼ Apoyo teórico / práctico

Simulación híbrida **UNIDAD III** — combina **eventos discretos** (arribos, fines de servicio) con una **variable continua** (temperatura) que evoluciona por una ecuación por tramos entre eventos.

Decaimiento exponencial (enfriamiento de Newton) — al descansar, la temperatura tiende a la ambiente como $T_a + (T - T_a) \cdot e^{(-k \cdot dt)}$: cuanto más caliente, más rápido baja. Es determinístico dado el historial de disparos.

Ruptura del supuesto markoviano **UNIDAD III** — M/M/c supone c servidores **siempre disponibles**; el cooldown viola eso, por lo que la fórmula queda como **cota** y la simulación es la que da el número real.

Cue de demo: mirar una torre roja apagándose es **ver bajar la capacidad efectiva** en tiempo real.

★ Apoyo material estudio

- **Variable de estado continua** **UNIDAD I** — describe el sistema entre eventos; acá la temperatura de cada torre, integrada por tramos.
- **Modelo dinámico** **UNIDAD I** — el estado evoluciona en el tiempo; combina lo discreto (cola) y lo continuo (temperatura).
- **Capacidad efectiva variable** **UNIDAD III** — cuando un servidor no está disponible, el sistema se comporta como uno con menos servidores: más cola y más bloqueo.

Martín Blasson

Wave 03 — validación y resultados + conclusiones. Slides **11 Validación** · **12 Rendimiento marginal** · **13 Óptimo económico** · **21 Conclusiones.** Demuestra que el modelo es correcto y responde “¿cuántas torres?”.

SLIDE 11 · WAVE 03 Validación: analítico vs. simulado

Referente: Martín

► PUNTOS A PRESENTAR

- Mostrar la tabla analítico vs. simulado (W_q , L_q , L , fuga P_b).
- Resaltar la **Ley de Little verificada** con error $\approx 1.2\%$ → el modelo es internamente consistente.
- Argumento doble: **sin temperatura**, $\text{sim} \approx \text{analítico}$ (motor validado); **con temperatura**, la brecha en la fuga ($0.16\% \rightarrow 4.0\%$) **mide el efecto** de lo que la fórmula no ve.

▼ Apoyo teórico / práctico

Validación de un modelo UNIDAD I/III — confrontar la salida de la simulación contra un referente conocido. Acá el referente son las **fórmulas cerradas M/M/c y M/M/c/K**.

Métricas de cola UNIDAD III — W_q espera media en cola; L_q n° medio en cola; L n° medio en el sistema; P_b probabilidad de bloqueo (fuga).

Erlang C / Erlang B — M/M/c (cola infinita) usa la fórmula de **Erlang C** para la probabilidad de esperar; M/M/c/K (capacidad finita) usa **Erlang B** para la probabilidad de bloqueo P_b .

Lectura clave de la brecha: si sólo difiere la fuga cuando enciendo la temperatura, entonces la temperatura es la causante — evidencia limpia para “por qué simular”.

Little: error $\approx 1.2\%$

P_b analít. **0.16%**

P_b sim. **4.0%**

★ Apoyo material estudio

- **M/M/1** UNIDAD III — el caso más simple, **1 torre**. Con $L = \text{n° medio en el sistema}$, $L_q = \text{n° medio en cola}$, $W = \text{tiempo medio en el sistema}$ y $W_q = \text{espera media en cola}$: $L = \rho / (1 - \rho)$, $L_q = \rho^2 / (1 - \rho)$, $W = 1 / (\mu - \lambda)$, $W_q = \rho / (\mu - \lambda)$ (todo se dispara cuando $\rho \rightarrow 1$).
- **M/M/c — fórmula de Erlang C** — lo mismo con **c torres**: da la probabilidad de que un cliente **tenga que esperar**, y de ahí L_q y W_q .
- **M/M/c/K — fórmula de Erlang B** — con capacidad finita K agrega la probabilidad de **bloqueo** P_b (llega con el sistema lleno → se pierde = fuga). La tasa que realmente entra es $\lambda_{ef} = \lambda(1 - P_b)$.
- **Validación por Ley de Little** — si en la corrida se cumple $L \approx \lambda_{ef} \cdot W$ (cantidad media = tasa de entrada × tiempo medio), el modelo es **internamente consistente**.

► PUNTOS A PRESENTAR

- Mostrar L_q vs. c y la mejora marginal ΔL_q por torre: **2.53** → **0.25** → **0.05** → **0.01**.
- Conclusión: la **1ª torre sobre $c_{\min}$ aporta ~89%** de toda la reducción de cola.
- **$c_{\min} = 2$** es el mínimo para estabilidad ($\rho < 1$); pasado el óptimo las torres quedan **ociosas** ($\rho \rightarrow 0$).

▼ Apoyo teórico / práctico

Rendimientos marginales decrecientes — cada servidor adicional reduce la cola, pero cada vez menos: la curva $L_q(c)$ es convexa y se aplan. Es un resultado típico de M/M/c.

$c_{\min}$ (estabilidad) **UNIDAD III** — el menor c que cumple $\rho = \lambda / (c\mu) < 1$. Con $\lambda = 0.40$ y $\mu = 0.25$: $c\mu > 0.40 \Rightarrow c \geq 2$. Con $c = 1$, $\rho > 1$ y la cola explota.

$$\Delta L_q(c) = L_q(c-1) - L_q(c) \text{ (ganancia de agregar la torre } c)$$

Servidor ocioso ($\rho \rightarrow 0$) — al sumar torres de más, ρ por servidor cae hacia 0: capacidad pagada que casi no se usa. Esto motiva el análisis económico del slide siguiente.

★ Apoyo material estudio

- **Comportamiento de $L_q(c)$** **UNIDAD III** — en M/M/c la cola cae al sumar servidores, con forma **convexa decreciente** (cada uno rinde menos).
- **Intensidad ofrecida** — $a = \lambda / \mu$ (Erlangs); para estabilidad hace falta **$c > a$** , es decir $\rho = a/c < 1$.
- **Análisis marginal** — comparar el beneficio de la c -ésima unidad contra su costo; principio que reaparece en el óptimo económico.

► PUNTOS A PRESENTAR

- Plantear el **costo total** como suma de costo de capacidad (torres) + costo de fugas.
- El primero crece con c , el segundo decrece con $c \rightarrow$ la suma es una **U** con mínimo interior: $c^* = 3$.
- Regla práctica: sumar una torre conviene **mientras el ahorro en fugas supere su costo**. Dimensionar a c^* , no al máximo.
- **Cue de demo:** agregar/sacar torres y ver cómo cambian cola y fugas en vivo.

▼ Apoyo teórico / práctico

Análisis de costo en colas UNIDAD III — el diseño óptimo balancea **costo de servicio** (más servidores) contra **costo de espera/pérdida** (clientes que esperan o se van).

$$CT(c) = c \cdot C_{\text{torre}} + C_{\text{fuga}} \cdot \lambda \cdot Pb(c)$$

Óptimo interior — como un término sube y el otro baja, el mínimo no está en los extremos sino “en el medio” ($c^*=3$). Es el punto donde el **ahorro marginal** en fugas iguala al **costo marginal** de una torre.

Mensaje de gestión: sobre-proveer (muchas torres) desperdicia capacidad; sub-proveer dispara las fugas. El valor del modelo es encontrar ese punto sin probar a ciegas.

★ Apoyo material estudio

- **Modelo de costos en colas** UNIDAD III — costo total = **costo de servicio** (crece con c) + **costo de espera/pérdida** (decrece con c).
- **Óptimo interior** — al sumar dos costos de sentido opuesto aparece un mínimo “en el medio”: el c que minimiza CT.
- **Costo óptimo de capacidad** UNIDAD III — objetivo central de la materia: el nivel de capacidad que **minimiza el costo global**, balanceando costo de servicio (más torres) contra costo de espera/pérdida (fugas). Dos costos contrapuestos $\Rightarrow$ mínimo intermedio (c^*).

► PUNTOS A PRESENTAR

- Recorrer los 6 takeaways: **estabilidad primero, retornos decrecientes** ($c^*=3$), **la temperatura importa, dimensionar al pico, prioridad = reasignar, gestión térmica**.
- Hilo conductor: cada conclusión sale de una slide anterior; sirve como resumen ejecutivo.
- Dar pie a Luciano para el cierre/demo.

▼ Apoyo teórico / práctico

De resultados a recomendaciones — el objetivo final de un estudio de simulación no es el número sino la **decisión**: cuántas torres, cómo priorizar, dónde invertir (enfriamiento vs. otra torre).

Gestión térmica vs. capacidad — recuperar disponibilidad mejorando el enfriamiento puede ser **más barato** que comprar otra torre: una palanca que sólo se ve porque se modeló la temperatura.

Síntesis — estabilidad ($\rho < 1$) es condición necesaria; el óptimo económico afina el cuánto; los experimentos (W4) ponen a prueba la robustez.

★ Apoyo material estudio

- **Síntesis de teoría de colas** **UNIDAD III** — estabilidad ($\rho < 1$), Ley de Little, rendimientos marginales, costos y disciplinas/prioridad.
- **Decisión bajo incertidumbre** **UNIDAD I** — la simulación convierte resultados estocásticos en recomendaciones de diseño.
- **Validez y límites del modelo** — las conclusiones valen dentro de los supuestos (λ , μ , K , modelo térmico); cambiarlos exige re-validar.

Luca Sucri

Wave 04 — experimentos, arquitectura y demo. Slides **14 Experimentos I · 15 Prioridad · 16 Arquitectura · 17 Frontend · 18 Diccionario visual · 19 HUD y escenarios · 20 Demo en vivo.** Cierra con rigor estadístico, ingeniería del sistema y la demostración.

SLIDE 14 · WAVE 04 Experimentos I: confianza y oleadas

Referente: Luca

► PUNTOS A PRESENTAR

- **Rigor:** con réplicas e **IC 95%** mostramos que **$c^*=3$ es robusto**, no un artefacto de la semilla.
- **Escenario no estacionario (oleadas $\lambda(t)$):** $c=2 \rightarrow 42\%$ de fuga vs. $c=4 \rightarrow 10\%$.
- Moraleja: **dimensionar al pico, no al promedio.**
- **Cue de demo:** lanzar una oleada en vivo y ver la cola saturarse y la base caer si faltan torres.

▼ Apoyo teórico / práctico

Réplicas independientes **UNIDAD III** — correr la simulación N veces con semillas distintas para estimar la **variabilidad** del resultado, no un único número “con suerte”.

Intervalo de confianza (IC 95%) — rango que, con 95% de confianza, contiene el verdadero valor. Si los IC de $c=3$ y $c=4$ no se solapan donde importa, la decisión es estadísticamente sólida.

Régimen estacionario vs. no estacionario — estacionario: λ constante, el sistema “se asienta”. **No estacionario:** $\lambda(t)$ varía (oleadas) y nunca llega al equilibrio; las fórmulas de régimen permanente no aplican y la **simulación es imprescindible**.

Dimensionar al pico: en sistemas con picos (call centers, oleadas), elegir c por el máximo de $\lambda(t)$ evita colapsos transitorios aunque el promedio “cierre”.

★ Apoyo material estudio

- **Análisis estadístico de la salida** **UNIDAD I/III** — media muestral, desvío, error estándar e **intervalo de confianza** (t-Student) sobre las réplicas.
- **Replicaciones independientes** — N corridas con semillas distintas; más réplicas $\Rightarrow$ IC más angosto (mayor precisión).
- **Transitorio vs. estacionario** — al arrancar vacío el sistema todavía “se está acomodando” (transitorio/warm-up) y sesga los promedios; se **descarta ese tramo inicial** antes de medir el régimen estacionario.
- **Sistemas terminantes vs. no terminantes** — **terminante:** tiene un fin natural (p. ej. un día de atención) y se mide ese horizonte; **no terminante:** corre indefinidamente y se mide el **equilibrio** de largo plazo. Acá usamos un horizonte fijo de 1200 s.

► PUNTOS A PRESENTAR

- Priorizar al enemigo **“fuerte”** baja su espera de **4.24 → 2.15 s**.
- El débil espera más, pero el **Wq total casi no cambia**: se **reparte** la espera entre clases, no se elimina (las torres nunca quedan ociosas con trabajo).
- Importante: la **fuga no cambia** con la prioridad — la fuga es por **bloqueo de capacidad**, no por espera.
- A igual media de servicio, **más variabilidad (varianza) ⇒ más fuga**.

▼ Apoyo teórico / práctico

Disciplinas de cola **UNIDAD III** — FIFO atiende por orden de llegada; con **prioridades** ciertas clases se atienden antes. Cambia el reparto de la espera entre clases.

Prioridad por “fuerza” — la disciplina no tiene por qué ser FIFO: atendiendo primero al enemigo **fuerte** (el que tarda más en morir) baja su espera. Es una **disciplina con prioridad**, tema de U3.

La espera total no cambia, se reparte — como las torres nunca quedan ociosas habiendo enemigos, el **tiempo de espera total** no cambia con la disciplina: al fuerte le sacamos espera y al débil se la sumamos. Sólo cambia **quién** espera.

Variabilidad del servicio — a **igual media** de servicio, cuanta más **varianza** (tiempos más dispares), mayor la cola (L_q/W_q) y más bloqueo. Por eso los “tipos de enemigo” con la misma media rinden distinto.

Distinción fina a remarcar: espera y fuga son cosas distintas — la prioridad mueve la **espera**; la fuga sólo baja con más **capacidad** (K o c).

★ Apoyo material estudio

- **Disciplinas de cola** **UNIDAD III** — además de **FIFO** (orden de llegada) y **LIFO**, está la atención por **prioridad** (con o sin desalojo del que ya está en servicio).
- **Prioridad al «fuerte»** — en vez de FIFO, atender primero a la clase que más tarda baja su espera. Es elegir la **disciplina** de la cola, no cambiar la capacidad.
- **Efecto de la varianza del servicio** — a **igual media**, cuanta más **varianza** (σ^2) en el tiempo de servicio, mayor la cola y más fuga. Es la intuición detrás de los “tipos de enemigo”.
- **La espera total se reparte, no cambia** — como las torres nunca quedan ociosas con enemigos esperando, cambiar el orden de atención **reparte** la espera entre clases pero no altera la espera media global.

► PUNTOS A PRESENTAR

- Mostrar el flujo: **RAG/doc** (fuente de verdad) → **Backend SimPy** (hace la matemática, exporta `output.json`) → **Frontend Godot** (sólo reproduce).
- **Cero acoplamiento en tiempo real:** Python calcula offline; Godot interpola. Corre en hardware modesto.
- El `output.json` v1.0 es el **único punto de contacto**, auto-auditado con 8 invariantes.

▼ Apoyo teórico / práctico

Separación cómputo / visualización — patrón de ingeniería de simulación: el **solver** (lógica estocástica, pesado) se separa del **visor** (liviano). Permite reproducibilidad y reuso del mismo resultado.

Contrato de datos (output.json v1.0) — esquema fijo que ambos lados respetan: eventos, muestras, layout y estadísticos. Garantiza que cambiar el visor no toque la matemática.

Auto-auditoría / invariantes — verificaciones automáticas (balance de enemigos, rangos válidos, consistencia temporal) que **verifican** el modelo programado antes de exportar — la parte de “¿lo programé bien?”.

Reproducibilidad de punta a punta: misma semilla ⇒ mismo `output.json` ⇒ misma reproducción visual. El azar queda totalmente bajo control.

★ Apoyo material estudio

- **Modelo conceptual → computacional** **UNIDAD I** — primero el modelo en papel (variables, relaciones), luego su implementación verificable.
- **Herramientas de simulación** **CLASE** — GPSS, AnyLogic y SimPy (la del TP); todas resuelven eventos discretos con distinto nivel de abstracción.
- **Números aleatorios comunes** **UNIDAD II** — usar la misma semilla entre escenarios reduce la varianza de la comparación (técnica de reducción de varianza).

► PUNTOS A PRESENTAR

- El front **lee** `output.json` **e interpola** la línea de tiempo: **cero cálculo** de simulación.
- Código de color: torres **azul→rojo** según temperatura, **gris** en cooldown; enemigos amarillos / rojos al fugarse; barra de vida de la base.
- Export web (HTML5/PWA) para hostear gratis (itch.io / GitHub Pages).
- Da pie: **a continuación, demo en vivo.**

▼ Apoyo teórico / práctico

Reproducción por interpolación — el visor no resuelve la dinámica; toma estados muestreados (temperatura, longitud de cola) en la grilla `dt_sample` y los **interpola** entre eventos para animar suave.

Lectura del contrato — `events[]` da la línea de tiempo (spawn, enqueue, start_service, kill, leak, overheat, cooldown_done); `samples[]` / `series` las variables continuas; `stats` / `analytical` alimentan el HUD.

Animación = traducción visual del modelo — cada color/posición corresponde a un estado del sistema de colas: ver “una torre roja apagándose” **es** ver bajar la capacidad efectiva.

★ Apoyo material estudio

- **Animación de la simulación** **UNIDAD I** — herramienta de **verificación** y comunicación: ver el modelo moverse ayuda a detectar errores y a explicarlo.
- **Modelo vs. animación** — el modelo calcula (estocástico); la animación sólo **muestra** estados ya calculados.
- **Muestreo de variables de estado** — se guardan estados en una grilla temporal y se interpolan para reproducirlos.

► PUNTOS A PRESENTAR

- Traducir el juego al modelo, elemento por elemento: **cueva** → **goblins** = llegadas **Poisson(λ)**; **torre** = servidor; **fila de goblins** = cola **FIFO** de capacidad **K**.
- **Recorrido del goblin hasta morir** = su **tiempo en el sistema**; **goblin rojo que llega al castillo** = la **fuga** (pérdida por bloqueo).
- **Temperatura** azul→roja + cooldown = la **capacidad que se degrada**.
- Insistir: es el **mismo** (M/M/c)(FIFO/K/ ∞) que validamos en la Wave 3, ahora jugable.

▼ Apoyo teórico / práctico

Mapeo modelo ↔ juego **UNIDAD III** — enemigo = **cliente**; torre = **servidor/canal**; fila = **cola**; fuga = **bloqueo** por capacidad finita. Es el mismo diccionario del slide del problema, ahora sobre la pantalla real.

La animación como verificación **UNIDAD I** — ver cada concepto moverse permite **chequear** que el modelo hace lo que dice y comunicarlo sin fórmulas.

Tiempo en el sistema (W) **UNIDAD III** — el recorrido completo del goblin (espera en cola + servicio) es exactamente W; la espera sola es Wq.

★ Apoyo material estudio

- **Vocabulario de colas** **UNIDAD III** — cliente, servidor, cola (capacidad K + disciplina), salida; cada uno tiene su contraparte visual.
- **L, Lq, W, Wq** **UNIDAD III** — n° en sistema / en cola y tiempo en sistema / en cola; en pantalla son "cuántos goblins hay" y "cuánto tardan".
- **Bloqueo (Pb)** **UNIDAD III** — el goblin que llega con la zona llena se fuga: es la probabilidad de pérdida que mide el modelo.

► PUNTOS A PRESENTAR

- Mostrar el **HUD** que pone la teoría en pantalla mientras corre: panel **Teoría vs Simulación** (ρ , L_q , W_q , L , W y fuga lado a lado).
- **Dimensionado c^*** : la tabla de **barrido de torres** resalta el óptimo según la carga (lo de la Wave 3, ahora interactivo).
- **Cola en el tiempo**: los picos del gráfico son las **ráfagas de llegadas** (aleatoriedad).
- **Cue de demo / interactivo**: con el **selector** cambiamos carga (λ), torres (c) y disciplina (FIFO/Prioridad) al instante y vemos reaccionar las métricas.

▼ Apoyo teórico / práctico

Validación en vivo UNIDAD I/III — poner **analítico vs. simulado** en pantalla es validar a la vista: si coinciden (sin temperatura), el motor está bien; si difieren (con temperatura), se ve el efecto.

Análisis de sensibilidad UNIDAD I — mover λ , c y la disciplina y observar cómo cambian cola, espera y fuga es explorar el modelo: qué pasa “si...”. El selector recarga corridas **precomputadas**.

c^* óptimo UNIDAD III — el barrido muestra el nivel de capacidad que minimiza el costo / equilibra cola y fugas; es el resultado de la Wave 3 vuelto interactivo.

★ Apoyo material estudio

- **Métricas en pantalla** UNIDAD III — λ , μ , c , K , ρ (entradas) y L , L_q , W , W_q , P_b (salidas), leídas en vivo.
- **Diseño de experimentos** UNIDAD I — comparar escenarios cambiando un factor por vez (carga, torres, disciplina) para aislar su efecto.
- **Régimen estacionario vs. ráfagas** UNIDAD III — el gráfico de cola en el tiempo muestra que con λ alta el sistema no “se asienta”: la cola crece y se satura.

► PUNTOS A PRESENTAR

- Correr el Tower Defense y guiar la mirada a **tres cosas: temperatura** (azul→rojo→cooldown), **la cola** (enemigos encolados FIFO), **la fuga** (zona llena → la base pierde vida).
- Controles: `Espacio` play/pausa • `←/→` velocidad • `R` reiniciar.
- Conectar lo que se ve con los conceptos: cada fenómeno visual es una métrica de la Wave 3.
- Tener el `output.json` recién generado copiado en el front y Godot ya abierto.

▼ Apoyo teórico / práctico

Qué demuestra la demo — cierra el argumento del TP: lo que la teoría predice (cola, espera, bloqueo) y lo que la temperatura agrega (cooldown $\Rightarrow$ más fuga) se ven **en movimiento** sobre el mismo sistema.

Anclaje conceptual en vivo — temperatura = variable continua que rompe el supuesto markoviano; cola = disciplina FIFO; fuga = bloqueo por capacidad finita K . Nombrarlos mientras ocurren refuerza todo lo anterior.

Plan B: si la demo falla, el deck tiene el `frontend_mock` y los gráficos; la validación numérica (W3) ya sostiene las conclusiones sin necesidad de la corrida en vivo.

★ Apoyo material estudio

- **Métricas a leer en pantalla** `UNIDAD III` — λ , μ , c , K , ρ y las salidas L , L_q , W , W_q y P_b (fuga).
- **Simulación híbrida** — combina una **variable continua** (temperatura, EDO por tramos) con **eventos discretos** (llegadas, servicios).
- **Teoría $\leftrightarrow$ observación** — cada fenómeno visible (cola que crece, torre en cooldown, fuga) es una métrica concreta de la materia.